

Санкт-Петербургский государственный университет
Факультет прикладной математики – процессов управления
ООП СВ.5005.2022 Прикладная математика, фундаментальная информатика и программирование

Шаго Павел Евгеньевич

*Математическое моделирование и реализация планировщика
процессов для гетерогенных процессорных архитектур*

Бакалаврская работа

Научный руководитель:

кандидат физ.-мат. наук, доцент

Корхов В. В.

Рецензент:

эксперт по архитектуре и сетевому стеку,

ООО НИЦ АЭРОДИСК, Анохин Ю. В.

Санкт-Петербург
2026

Цель работы

разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению со штатным планировщиком ядра Linux

Предмет исследования

методы планирования, учитывающие различие вычислительной мощности ядер

Гетерогенные процессоры в современных устройствах

Сегмент	Примеры	Массово	Сегодня
Смартфоны	Apple A, Snapdragon, Dimensity, Exynos, Tensor	2013	все актуальные
Ноутбуки	Apple M, Intel Core Ultra, Ryzen AI, Snapdragon X	2020	все актуальные
Настольные ПК	Intel Core 12+, Intel Core Ultra	2021	частично
Серверы	NVIDIA GB10	—	экспериментально

Тренд: гетерогенные P/E-архитектуры, возникшие в мобильных SoC, стали нормой для процессоров ПК и ноутбуков. Серверные CPU пока остаются однородными

Временные кванты Р/Е-ядер процессора представляют собой **недолговечный ресурс** (неиспользованный квант теряется безвозвратно). Их распределение моделируется динамическим аукционом VCG с задачами-агентами типа $\theta_i = (v_i, l_i)$, максимизирующим социальное благосостояние

Задачи работы

- 1 Проанализировать и определить наиболее перспективный подход к реализации планировщика
- 2 Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора
- 3 Реализовать разработанный алгоритм и собрать воспроизводимый набор бенчмарков
- 4 Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым планировщиком Linux EEVDF

Возможные решения

- 1 Прямая модификация планировщика в ядре Linux
- 2 Загружаемый модуль ядра Linux
- 3 Пользовательский агент gh0St
- 4 eBPF-программа поверх sched_ext

Критерий	В ядре	Модуль	gh0St	sched_ext
Без пересборки ядра	—	+	±	+
Устойчивость к ошибкам политики	—	—	+	+
Низкие накладные расходы	+	+	—	±
Поддерживаемый интерфейс замены	—	—	±	+
Переносимость между версиями ядра	—	—	±	+

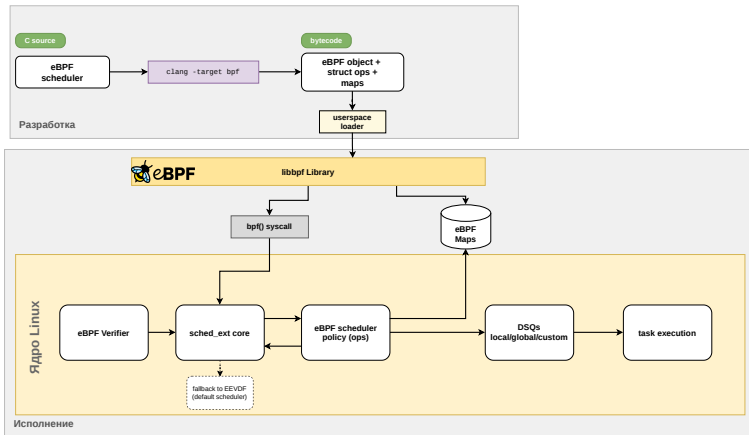


Рис. 1. Механизм работы sched_ext

DSQ (dispatch queue)

- SCX_DSQ_LOCAL
- SCX_DSQ_GLOBAL
- пользовательские DSQ

Цикл планирования

- select_cpu
- enqueue
- dispatch
- running / stopping

Аппаратная платформа

- Ядра: $\mathcal{P} = \{P_1, \dots, P_{K_P}\}$,
 $\mathcal{E} = \{E_1, \dots, E_{K_E}\}$
- Дискретное время $t = 1, 2, 3, \dots$
- Замедление на E-ядре:
 $\sigma = \eta_P / \eta_E > 1$
- Общий ISA (любая задача может быть выполнена на любом ядре)

Задачи как агенты

- Тип $\theta_i = (v_i, l_i)$ (оценка и длина)
- Полезность как маргинальный вклад $U_i = W(s^t) - W_{-i}(s^t) \geq 0$
- Бюджет $B_i > 0$ по классу обслуживания:

Реального времени	B_{RT}
Интерактивный	B_{int}
Пакетный	B_{batch}
Фоновый	B_{bg}

На E-ядре задача длиной l_i выполняется за $m_E(l_i) = \lceil l_i \cdot \sigma \rceil$ квантов. Ценность v_i начисляется только при полном выполнении контракта (допущение атомарности контракта).

Состояние $s^t = (\mathbf{x}^t, \mathbf{b}^t, \Theta^t, \mathcal{N}^t)$

- $\mathbf{x}^t$ – остаточные длительности контрактов на ядрах
- $\mathbf{b}^t$ – остаточные бюджеты активных задач
- Θ^t – типы текущих задач
- $\mathcal{N}^t$ – множество активных задач

Действие $a_i^t = (a_{i,P}^t, a_{i,E}^t) \in \{0, 1\}^2$

$$a_{i,P}^t + a_{i,E}^t \leq 1, \quad \forall i$$

$$\sum_i a_{i,P}^t \leq K_P^t, \quad \sum_i a_{i,E}^t \leq K_E^t$$

Функция перехода $s^{t+1} = G(s^t, a^t, \mathcal{N}^{t+1})$

$$x_k^{t+1} = \begin{cases} m_\kappa(l_i) - 1, & \text{ядро } k \text{ занято } i \\ \max(x_k^t - 1, 0), & \text{иначе} \end{cases}$$

$$b_i^{t+1} = b_i^t - p_i^t$$

$m_P(l) = l$, $m_E(l) = \lceil l\sigma \rceil$ – длина контракта

Социальное благосостояние –
уравнение Беллмана, $\delta \in (0, 1)$ –
дисконт:

$$W(s^t) = \max_{a^t} \left[\sum_i \phi_i^a + \delta \mathbb{E}[W(s^{t+1})] \right]$$

$\phi_i^a = a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)$ (см. след.
слайд)

Эффективная ценность задачи i на ядре типа $\kappa \in \{P, E\}$

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \sigma \rceil$$

Оптимальное распределение (максимизация W)

$$a^t = \arg \max_a \sum_i [a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)] + \delta \mathbb{E}[W(s^{t+1})]$$

$\delta \in [0, 1)$ – дисконт будущего благосостояния

Платёж VCG задачи i

$$p_i^t = W_{-i}(s^t) - (W(s^t) - \phi_\kappa(\theta_i))$$

$W(s^t)$, $W_{-i}(s^t)$ – оптимальное социальное благосостояние с i и без неё. Задача i платит ровно то, что теряют остальные агенты из-за её присутствия (pivot rule).

Свойства механизма

- Индивидуальная рациональность $u_i \geq 0$
- Бюджетное ограничение $p_i \leq B_i^t$
- Операционный антистимул к завышению v_i (платёж $\rightarrow$ бюджет)

Решение аукциона – задача направляется на кластер $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}(\theta_i)$
(пересматривается при пробуждении задачи)

DSQ_P

$$\phi_P \geq \phi_E$$

Эффективная ценность
выше на P-ядре.
Задача выигрывает
от пиковой частоты
производительного ядра

DSQ_E

$\phi_E > \phi_P$ либо
асимметричное
перенаправление
коротких задач из
перегруженного
P-кластера

$$Q_P \geq K_P, \quad Q_P K_E > Q_E K_P$$

DSQ_Starved

$$B_i^t < B_i^{\max}/10$$

Бюджет почти исчерпан.
Низший приоритет
диспетчеризации,
обслуживается
обоими кластерами
после пополнения

Бюджет – корзина токенов. $B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$; пополнение за время простоя
 $\Delta B_i = \min(\Delta t_{\text{idle}}, T_{\text{max}}) \cdot w_i / D_{\text{rep}}$ обеспечивает устойчивое обслуживание долгоживущих задач

Система

Ядро Linux 7.0,
Arch Linux
CPU: Intel Core
Ultra 7 265K
8P + 12E, без HT

Сравнение

Linux EEVDF
(эталон)
scx_EEVDF
scx_LAVD
scx_A1349 (VCG)

Бенчмарки

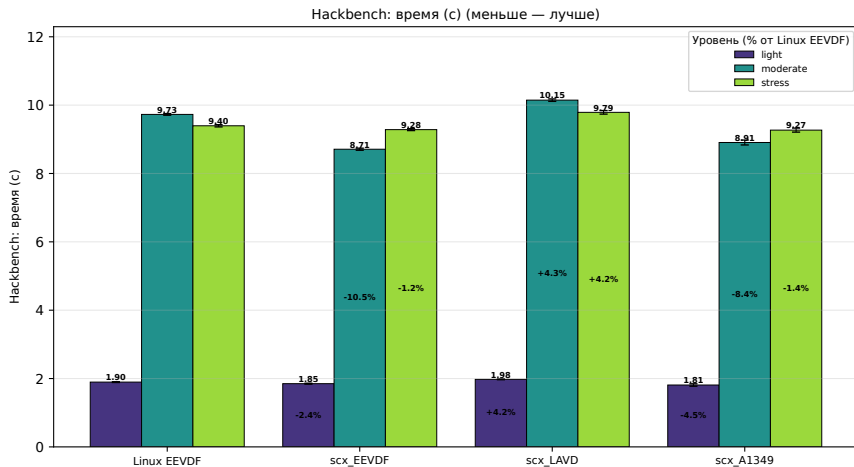
- **hackbench** – группы процессов, обмен короткими сообщениями. Массовое порождение короткоживущих задач, стресс пробуждений и миграций
- **sysbench** – oltp_read_only, транзакции чтения к PostgreSQL. Потoki блокируются на ожидании ответа БД
- **schbench** – задержки планирования, веб-нагрузка. Запрос – две фазы сна и фаза матричных вычислений

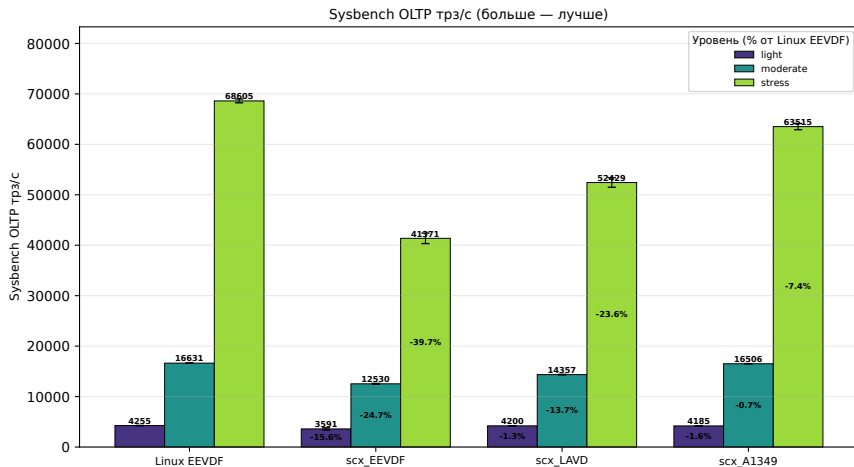
Метрики

hackbench: время (с)↓
sysbench: TPS↑
schbench: RPS↑,
p99 / p99.9 (мкс)↓
RAPL package: энергия (Дж)↓

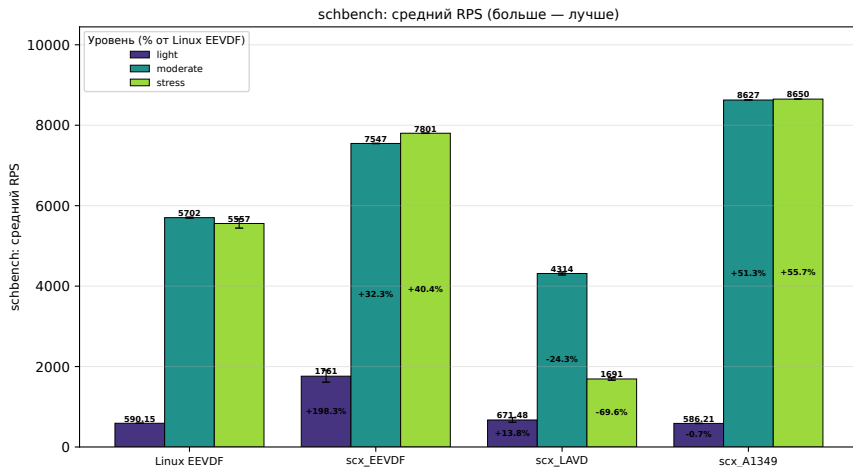
3 уровня
нагрузки,
 $N = 6$, 95% ДИ

Результаты. Hackbench

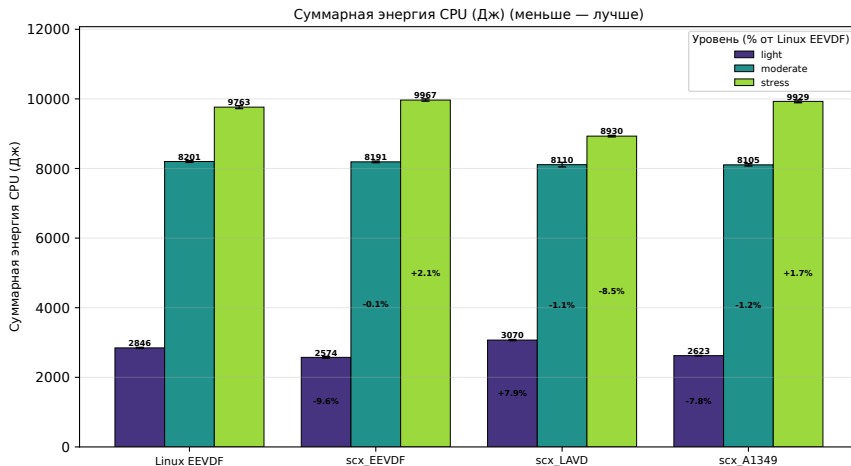




Результаты. Schbench



Результаты. Энергопотребление



- Распределение слотов P/E-ядер формализовано как динамический аукцион VCG: состояние, действие, эффективная ценность ϕ_{κ} , платёж по pivot-правилу. Обоснованы стимульная совместимость в пределах класса, индивидуальная рациональность и бюджетное ограничение
- Механизм реализован как BPF-планировщик scx_A1349 поверх sched_ext: три кластерные DSQ, маршрутизация $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}$, бюджет – корзина токенов
- На Intel Core Ultra 7 265K (8P+12E) scx_A1349 быстрее штатного EEVDF на hackbench при всех уровнях нагрузки (+4,5 / +8,5 / +1,4 %) и лидирует по RPS schbench при умеренной и стрессовой нагрузке (+51 / +56 %)
- При стрессе хвостовая задержка запроса p99.9 составляет 24,5 мс против $\approx 1,7$ с у штатного EEVDF. На sysbench все sched_ext-планировщики уступают штатному EEVDF из-за накладных расходов фреймворка

Спасибо за внимание!

Пример: $K_P = K_E = 1$, одно свободное ядро типа κ

Победитель:

$$i^* = \arg \max_{i \in \mathcal{N}^t, B_i^t > 0} \phi_\kappa(\theta_i) + \delta^{m_\kappa(l_i)} \bar{W}_\kappa$$

где $m_P(l) = l$, $m_E(l) = \lceil l \cdot \sigma \rceil$

Платёж победителя:

$$p_{i^*} = \phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa$$

где j — задача на втором месте, $\bar{W}_\kappa$ — ожидаемое будущее благосостояние кластера.

$\phi_P(\theta_i) > \phi_E(\theta_i)$ выполняется не всегда: при малом v_i и большом l_i E-ядро может дать большую эффективную ценность ($c_E \cdot \lceil l_i \sigma \rceil < c_P \cdot l_i$ при $\sigma < \gamma$). Оптимальная стратегия нетривиальна.